

# MEDICAL AI SEVERITY SYSTEM

## Team Onboarding Guide

What we are building · How it works · What you need to do

**7**  
Pipeline Layers

**14**  
Data Types

**3**  
Diseases (Phase 1)

**1**  
Year Project

For Beginners · No Prior AI Experience Required

# What Are We Building?

We are building a **Medical AI Severity System** — a software tool that a doctor or nurse types a patient's symptoms and test results into, and gets back a clear answer: **how sick is this patient, and why?**

## SIMPLE ANALOGY

Think of it like a very experienced doctor's assistant. You describe the patient to it ("high blood sugar, overweight, 52 years old"), and it gives you back: **"Severity Score: 72/100 — High Risk for Diabetes"** plus an explanation citing medical research journals, so the real doctor can make a faster, better decision.

## The Three Diseases We Start With

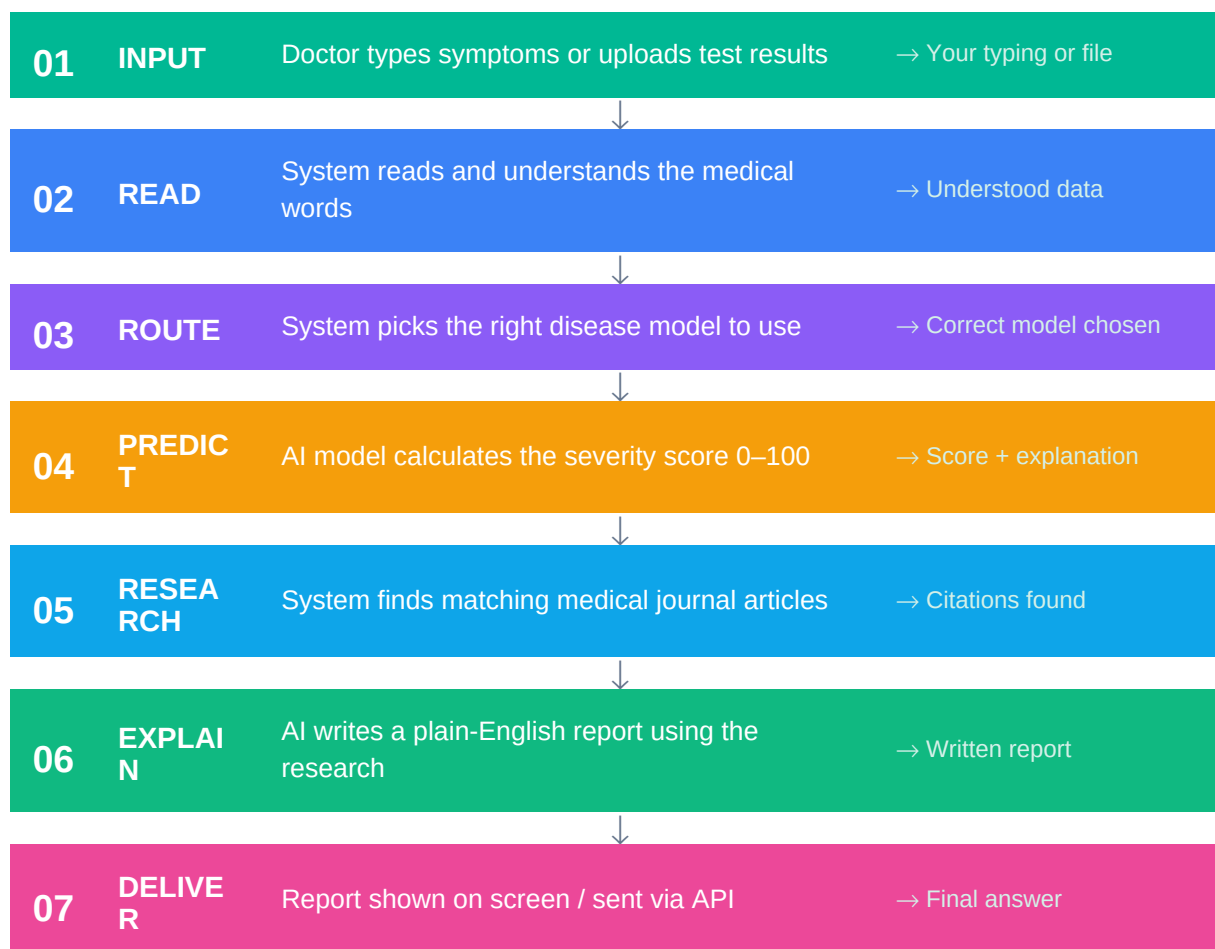
|                  |                                                                         |
|------------------|-------------------------------------------------------------------------|
| ■ Diabetes       | High blood sugar over time. Very common, often undetected early.        |
| ♥■ Heart Disease | Problems with the heart and arteries. Leading cause of death worldwide. |
| ■ Pneumonia      | Lung infection causing breathing difficulty. Needs fast treatment.      |

## What Does "Severity Score 0–100" Mean?

| Score    | Label    | What It Means                        | What Happens                   |
|----------|----------|--------------------------------------|--------------------------------|
| 0 – 24   | Low      | Patient shows minimal risk signs     | Monitor, no urgent action      |
| 25 – 49  | Moderate | Some concerning indicators present   | Follow-up recommended          |
| 50 – 74  | High     | Multiple strong risk factors present | Doctor review needed soon      |
| 75 – 100 | Critical | Urgent clinical indicators           | Immediate physician referral ■ |

# How It Works — The Simple Version

Think of the system as a **factory with 7 rooms (layers)**. Patient information goes into Room 1 and comes out of Room 7 as a finished medical report. Each room does one specific job.



## Real Example — What Happens With One Patient

|         |                                                                             |
|---------|-----------------------------------------------------------------------------|
| Input   | Nurse types: "Patient is 52, glucose 210 mg/dL, BMI 34, frequent urination" |
| Layer 2 | System reads: glucose=210, age=52, bmi=34, domain=DIABETES                  |
| Layer 3 | Selects the Diabetes XGBoost model from the registry                        |
| Layer 4 | Model outputs: Severity Score = 74/100 (High). Top risk: high glucose.      |
| Layer 5 | Finds 3 medical papers about high glucose + diabetes risk from database     |

|                |                                                                                                 |
|----------------|-------------------------------------------------------------------------------------------------|
| <b>Layer 6</b> | Writes: "Patient shows High severity (74/100). Primary driver: elevated glucose (210 mg/dL)..." |
| <b>Layer 7</b> | Sends the report to the screen. Doctor sees severity gauge + explanation + paper citations.     |

# The 7 Layers — Detailed Breakdown

Each layer below shows exactly what goes in, what comes out, and which tools are used.

| LAYER 01    Multimodal Input Router |                                                                                                                                                                                                                                                                            |
|-------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| What it does                        | Accepts anything — typed text, lab result files (CSV), medical scan images (DICOM), ECG heart-trace files, or even audio. Checks if the data is valid and routes it to the right processor. If important data is missing, it asks the user to provide it (dialogue agent). |
| Goes IN                             | Free-text symptoms OR CSV files OR DICOM images OR ECG waveform files                                                                                                                                                                                                      |
| Comes OUT                           | A clean, typed "payload" ready for the next layer                                                                                                                                                                                                                          |
| Tools used                          | Python file readers, DICOM library, missing-data chat agent                                                                                                                                                                                                                |

| LAYER 02    Clinical Entity Extraction (NER) |                                                                                                                                                                                                                                                                        |
|----------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| What it does                                 | Reads the text and pulls out the important medical facts: disease names, lab values, medications, body parts mentioned. Also detects negation ("no chest pain" means chest pain is absent). For images, normalises pixel brightness. For ECG, detects heartbeat peaks. |
| Goes IN                                      | Raw patient text like "glucose 210, BMI 34, frequent urination"                                                                                                                                                                                                        |
| Comes OUT                                    | Structured data: {glucose: 210, bmi: 34, domain: 'diabetes', missing: ['insulin']}                                                                                                                                                                                     |
| Tools used                                   | BioBERT (medical language AI), NegEx algorithm, NeuroKit2 for ECG                                                                                                                                                                                                      |

| LAYER 03    Model Registry & Router |                                                                                                                                                                                                              |
|-------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| What it does                        | Looks at the domain (diabetes? heart? pneumonia?) and picks the correct ML model. Think of it like a switchboard — the right expert gets the call. New diseases can be added without changing anything else. |
| Goes IN                             | Domain name from Layer 2 (e.g. 'diabetes')                                                                                                                                                                   |
| Comes OUT                           | The correct XGBoost model loaded and ready                                                                                                                                                                   |
| Tools used                          | Python class registry, joblib model loader                                                                                                                                                                   |

| LAYER 04     | Severity Inference + SHAP                                                                                                                                                                                                                                                                                       |
|--------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| What it does | The ML model runs the numbers and outputs a probability of disease, which gets converted to a 0–100 score. SHAP (SHapley Additive exPlanations) then calculates <b>which individual features caused the score</b> — like a receipt showing what pushed the score up or down. This is critical for doctor trust. |
| Goes IN      | Feature array: [210, 34, 52, 72, ...] (all the patient's numbers)                                                                                                                                                                                                                                               |
| Comes OUT    | severity_score=74, top_features=['glucose', 'bmi', 'age'], shap_values={...}                                                                                                                                                                                                                                    |
| Tools used   | XGBoost, SHAP library, calibration math                                                                                                                                                                                                                                                                         |

| LAYER 05     | RAG Knowledge Base                                                                                                                                                                                                                                                                                                                                              |
|--------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| What it does | RAG = Retrieval-Augmented Generation. We stored 500+ medical journal abstracts in a "smart library" (vector database). The SHAP top features become search terms. So if glucose was the top risk factor, it searches for papers about high glucose + diabetes. A faithfulness checker ensures the AI only cites what it actually found — no made-up references. |
| Goes IN      | SHAP query string like 'diabetes elevated glucose high BMI risk'                                                                                                                                                                                                                                                                                                |
| Comes OUT    | Top 3–5 relevant medical paper excerpts + their sources                                                                                                                                                                                                                                                                                                         |
| Tools used   | ChromaDB vector database, sentence-transformers, RAGAS faithfulness checker                                                                                                                                                                                                                                                                                     |

| LAYER 06     | LLM Report Generator + Safety Rails                                                                                                                                                                                                                                                                                                                |
|--------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| What it does | An AI language model (Llama 3, free via Groq) writes the final plain-English report using the score + SHAP explanation + retrieved papers. Safety rules are hardcoded: never suggest specific drug doses, always say 'consult a doctor', auto-flag Critical scores. The report is grounded — it can only cite papers that were actually retrieved. |
| Goes IN      | severity_score=74, top_features, 3 paper excerpts                                                                                                                                                                                                                                                                                                  |
| Comes OUT    | A written explanation: summary, risk factors, evidence, disclaimer                                                                                                                                                                                                                                                                                 |
| Tools used   | Llama-3-8B via Groq API (free), LangChain prompt template, safety filters                                                                                                                                                                                                                                                                          |

| LAYER 07    API + User Interface |                                                                                                                                                                                                                                                      |
|----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| What it does                     | FastAPI creates a web endpoint (/predict) that any system can call. Streamlit creates the visual interface: severity gauge, bar chart of SHAP values, clickable citations, and the full report. The whole system runs in Docker for easy deployment. |
| Goes IN                          | The final structured report + score from Layer 6                                                                                                                                                                                                     |
| Comes OUT                        | JSON response for developers / visual dashboard for doctors                                                                                                                                                                                          |
| Tools used                       | FastAPI, Streamlit, Docker, GCP Cloud Run                                                                                                                                                                                                            |

# Inputs & Outputs — Complete Reference

This page tells you exactly **what data the system accepts** and **what it gives back**.

## What Can You Feed Into the System? (Inputs)

| Input Type               | Example                                    | Format        | Phase     |
|--------------------------|--------------------------------------------|---------------|-----------|
| Typed symptoms           | "Patient has high blood sugar, 52 yrs old" | Plain text    | ■ Phase 1 |
| Lab results (blood test) | Glucose=210, HbA1c=8.2, BMI=34             | CSV or typed  | ■ Phase 1 |
| Chest X-ray image        | Image of patient's chest                   | DICOM / PNG   | ■ Phase 1 |
| ECG (heart trace)        | 12-lead heart rhythm recording             | EDF / XML     | ■ Phase 2 |
| Brain MRI scan           | 3D brain volume scan                       | NIfTI / DICOM | ■ Phase 2 |
| Lung sounds (audio)      | Recorded stethoscope audio                 | WAV file      | ■ Phase 3 |
| Genomics                 | Patient DNA variant file                   | VCF / FASTQ   | ■ Phase 3 |

## What Does the System Give Back? (Outputs)

| Output Field            | Example Value                               | Who Uses It                        |
|-------------------------|---------------------------------------------|------------------------------------|
| <b>severity_score</b>   | 74.0 (a number 0–100)                       | Doctors, nurses, triage teams      |
| <b>severity_label</b>   | "High" (Low / Moderate / High / Critical)   | Quick visual indicator             |
| <b>confidence</b>       | 88% (how sure the model is)                 | Doctors evaluating trustworthiness |
| <b>top_risk_factors</b> | ["glucose", "bmi", "age"]                   | Shows WHAT is causing the risk     |
| <b>explanation</b>      | Full paragraph report citing medical papers | Doctors reading the reasoning      |
| <b>citations</b>        | ["ADA Standards of Care 2024", ...]         | Doctors verifying sources          |
| <b>disclaimer</b>       | Standard medical safety warning             | Legal and ethical compliance       |

# Team Roles — Who Builds What

---

Divide the work across 4 roles. Each person owns specific layers.

| ML Engineer                                                                         |                                                                                                                                                                                                                                                          | Owens: Layers 3 & 4    |
|-------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------|
|    | <ul style="list-style-type: none"><li>• Download Kaggle datasets (diabetes, heart, pneumonia)</li><li>• Train XGBoost models in Kaggle Notebooks (free GPU)</li><li>• Implement SHAP explainer</li><li>• Save models as .joblib files → GitHub</li></ul> |                        |
| Backend / Pipeline Engineer                                                         |                                                                                                                                                                                                                                                          | Owens: Layers 1, 2 & 7 |
|    | <ul style="list-style-type: none"><li>• Build the input router (accepts text, CSV, images)</li><li>• Build ClinicalNER processor</li><li>• Build FastAPI /predict endpoint</li><li>• Write unit tests for each tool</li></ul>                            |                        |
| RAG / Knowledge Engineer                                                            |                                                                                                                                                                                                                                                          | Owens: Layer 5         |
|  | <ul style="list-style-type: none"><li>• Download PubMed abstracts (script provided)</li><li>• Set up ChromaDB vector database</li><li>• Build semantic retriever</li><li>• Implement RAGAS hallucination checker</li></ul>                               |                        |
| Frontend / Integration Engineer                                                     |                                                                                                                                                                                                                                                          | Owens: Layers 6 & 7    |
|  | <ul style="list-style-type: none"><li>• Build Streamlit UI (severity gauge, SHAP chart)</li><li>• Connect LLM report generator (Groq Llama 3)</li><li>• Implement safety rails</li><li>• Deploy to Streamlit Cloud (free)</li></ul>                      |                        |

# Week 1 — What To Do First

Do these steps in order. Do not skip ahead. Each step builds on the previous one.

|         |                                                                                                                                                                                                                                                                             |
|---------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Day 1   | <b>Create the GitHub repository</b><br>Go to <a href="https://github.com">github.com</a> → New Repository → name it 'medical-ai-severity' → clone to your laptop. This is where ALL code lives. Everyone pushes here.                                                       |
| Day 1   | <b>Create the folder structure</b><br>Create folders: <code>src/</code> , <code>data/</code> , <code>notebooks/</code> , <code>models/</code> , <code>tests/</code> . Add a <code>.gitignore</code> so large data files don't get committed.                                |
| Day 2   | <b>Install Python and packages</b><br>Run: <code>pip install xgboost shap chromadb sentence-transformers fastapi uvicorn langchain langchain-groq streamlit</code>                                                                                                          |
| Day 2   | <b>Run the smoke test</b><br>Download <code>medical_ai_e2e.py</code> from our repo. Run: <code>python medical_ai_e2e.py</code> — you should see all tests pass. This proves your environment works.                                                                         |
| Day 3–4 | <b>Download Kaggle datasets</b><br>Go to <a href="https://kaggle.com">kaggle.com</a> . Download: Pima Diabetes dataset, Heart Disease UCI dataset, Chest X-Ray Pneumonia dataset. Put them in <code>data/raw/</code>                                                        |
| Day 4–5 | <b>First Jupyter notebook</b><br>Open <code>notebooks/01_eda_diabetes.ipynb</code> . Load the diabetes CSV. Print <code>head()</code> , <code>describe()</code> , check for missing values. Plot the glucose histogram. This is your first EDA (Exploratory Data Analysis). |
| Day 5   | <b>Get your free Groq API key</b><br>Go to <a href="https://console.groq.com">console.groq.com</a> → Sign up (free) → Create API key. Add to <code>.env</code> file: <code>GROQ_API_KEY=your_key_here</code> . This gives you free Llama 3 access.                          |

## Key Tools — Quick Reference

| Tool           | What it is             | Why we use it                          | Cost        |
|----------------|------------------------|----------------------------------------|-------------|
| <b>XGBoost</b> | Machine learning model | Predicts disease severity from numbers | <b>Free</b> |

|                       |                        |                                                   |      |
|-----------------------|------------------------|---------------------------------------------------|------|
| <b>SHAP</b>           | Explainability library | Shows WHICH features caused the prediction        | Free |
| <b>ChromaDB</b>       | Vector database        | Smart search through medical papers               | Free |
| <b>LangChain</b>      | AI orchestration       | Connects all the AI pieces together               | Free |
| <b>Groq + Llama 3</b> | Free AI language model | Writes the final report in plain English          | Free |
| <b>FastAPI</b>        | Web API framework      | Creates the /predict endpoint other apps can call | Free |
| <b>Streamlit</b>      | UI framework           | Builds the visual dashboard in Python             | Free |
| <b>Kaggle</b>         | Data + GPU platform    | Download datasets, train models for free          | Free |

# Glossary — Technical Words Explained Simply

---

|                        |                                                                                                                                                                    |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Pipeline</b>        | The full sequence of steps from input to output. Like an assembly line.                                                                                            |
| <b>Layer</b>           | One step in the pipeline. Each layer does one specific job.                                                                                                        |
| <b>XGBoost</b>         | A type of machine learning model that is very good at working with numbers in tables. Like a very smart calculator that learned from thousands of patient records. |
| <b>SHAP</b>            | A method that shows WHY the model gave a certain score. It says 'glucose was responsible for +30 points, BMI for +20 points...'                                    |
| <b>RAG</b>             | Retrieval-Augmented Generation. The system looks up real medical papers before writing the report, so it doesn't make things up.                                   |
| <b>Vector Database</b> | A special database that stores text as numbers (vectors) so you can search by meaning, not just by exact words.                                                    |
| <b>ChromaDB</b>        | The specific vector database we use. Stores medical paper embeddings.                                                                                              |
| <b>LLM</b>             | Large Language Model. The AI that reads and writes text, like GPT-4 or Llama 3.                                                                                    |
| <b>LangChain</b>       | A Python library that helps connect LLMs to tools, databases, and other code.                                                                                      |
| <b>FastAPI</b>         | A Python library for creating web APIs. It lets other programs call our AI via HTTP.                                                                               |
| <b>Streamlit</b>       | A Python library that turns Python code into a web dashboard with no HTML/CSS needed.                                                                              |
| <b>DICOM</b>           | The file format medical scanners (CT, MRI, X-ray) use to save images.                                                                                              |
| <b>FHIR</b>            | A standard format for sharing patient health records electronically.                                                                                               |
| <b>EDA</b>             | Exploratory Data Analysis. Opening a dataset and looking at it before training. Checking for missing values, plotting distributions, understanding what you have.  |
| <b>Severity Score</b>  | Our system's output: a number 0–100 saying how serious a patient's condition is.                                                                                   |
| <b>RAGAS</b>           | A library that checks if our AI report is grounded in real retrieved evidence (not hallucinated).                                                                  |
| <b>Hallucination</b>   | When an AI makes up a fact that sounds true but isn't. We prevent this using RAG + RAGAS.                                                                          |
| <b>Groq</b>            | A free AI API service. We use it to run Llama 3 (a language model) for free.                                                                                       |
| <b>NER</b>             | Named Entity Recognition. AI reading text and labelling the important parts: drug names, lab values, disease names, etc.                                           |
| <b>Smoke Test</b>      | A quick test to check everything is connected and working, before testing in detail.                                                                               |

# Project Checklist — Track Your Progress

---

## Phase 1 — Foundation (Weeks 1–4)

- GitHub repo created with correct folder structure
- All team members can clone and push to repo
- Python environment set up, all packages installed
- Smoke test runs and passes (python medical\_ai\_e2e.py)
- 3 Kaggle datasets downloaded to data/raw/
- EDA notebooks completed for all 3 diseases
- Groq API key obtained and tested

## Phase 2 — ML Models (Weeks 5–10)

- DiabetesModel trained on Kaggle (AUC > 0.85)
- HeartDiseaseModel trained on Kaggle (AUC > 0.85)
- PneumoniaModel trained on Kaggle (AUC > 0.85)
- SHAP explainer working for all 3 models
- Model registry routing correctly (smoke test passing)
- All .joblib files saved to models/ and pushed to GitHub

## Phase 3 — RAG Pipeline (Weeks 11–16)

- ingest\_pubmed.py script working
- 500+ abstracts per disease ingested into ChromaDB
- Semantic retrieval returning relevant papers
- SHAP → RAG query bridge working
- RAGAS faithfulness score > 0.8
- Hallucination guard blocking fabricated citations

#### Phase 4 — API & UI (Weeks 17–20)

- FastAPI /predict endpoint working
- Streamlit UI showing severity gauge
- SHAP bar chart visible in UI
- Citations displayed in UI
- Safety disclaimer appearing on all outputs
- End-to-end test passing with all 3 diseases

#### Phase 5 — Deployment (Weeks 21+)

- Docker container builds without errors
- Deployed to Streamlit Community Cloud
- API deployed to Render.com (free tier)
- Demo video recorded
- README.md complete with setup instructions
- Research paper outline started

This is a 1-year project. Phase 1 is what matters most right now. Build it layer by layer, test at every step, and commit to GitHub daily. Every team member should be able to explain what each layer does in plain English before writing any code.